

Freestanding Library: inout expected span

Document number: P2833R1

Date: 2023-08-19

Reply-to: Ben Craig <ben dot craig at gmail dot com>

Audience: Library Evolution Working Group

Changes from previous revisions

Changes from R0

- Reworked wording based on changes in P2407R5
- Made feature test macros `// freestanding`
- Discuss `P2821 span::at` merge conflict

Introduction

This proposal is part of a group of papers aimed at improving the state of freestanding. It adds facilities that are largely unrelated to each other, but didn't fit into a better category. Most of these facilities were added in C++23, and missed the last sweep of freestanding utilities.

At a high level, this proposal adds the following facilities:

- `out_ptr`
- `inout_ptr`
- Most of `expected`
- `span`
- `mdspan`

Motivation and Scope

On hosted implementations, the included facilities don't directly require heap allocations, don't use system calls, and don't require exceptions. The facilities are generally useful. This combination keeps the burden low for standard library implementations to single source these facilities as both hosted and freestanding.

Design

All overloads of `expected::value` are deleted, as they can throw exceptions. This follows the precedent from P2407 "Freestanding Library: Partial Classes".

The declaration of `shared_ptr` is included, but the template class is left undefined. This allows `out_ptr` and `inout_ptr` to disallow certain `shared_ptr` usages in the same way in hosted and freestanding. This approach is less specification work and less implementation work, while still providing reasonable diagnostics to users if they mistakenly use `shared_ptr` on a minimal freestanding implementation. This approach still allows non-minimal freestanding implementations to provide a `shared_ptr` implementation that will interact with `out_ptr` and `inout_ptr` the same as they do on hosted implementations.

This paper mostly follows the guide lines as specified in [P2198R6] "Freestanding Feature-Test Macros and Implementation-Defined Extensions". However, `__cpp_lib_freestanding_mdspan` is added instead of bumping `__cpp_lib_mdspan` because I suspect the `mdspan` authors will be bumping the `__cpp_lib_mdspan` macro frequently in the C++26 cycle. When multiple orthogonal papers bump the same feature test macro, it becomes difficult for users to test for the specific features they care about.

The in flight paper [P2821] "span.at()" paper adds `at()` to `span`. The `at()` function originates exceptions, so it will not be required to be present in a freestanding implementation. This follows the precedent set by `array` in P2407 "Freestanding Library: Partial Classes".

Experience

[P0323R12 std::expected] cites usage experience on bare metal systems without exceptions enabled. [P0009R18 MDSPAN] has a reference implementation that is usable with CUDA. mdspan uses span in the implementation and interface, so the CUDA experience extends to span as well.

The author has prototyped the necessary out_ptr and inout_ptr changes with MSVC's STL. This involved #if'ing out weak_ptr, as well as the definitions of shared_ptr facilities. The tests worked as is, except for the ones specifically testing shared_ptr.

Wording

This paper's wording is based on the current working draft, [N4950], and it assumes that [P2407R5] has been applied.

Change in [freestanding.item]

Add a new paragraph to [freestanding.item].

? Class template declarations followed by a comment that include *freestanding-deleted* are *freestanding deleted class template declarations*. On freestanding implementations, it is implementation-defined whether each entity introduced by a freestanding deleted class template declaration is undefined or whether the requirements are the same as the corresponding requirements for a hosted implementation.
[Note: An instantiation of an undefined template yields an incomplete class type (temp.inst#2.sentence-4). -end note]
[Example:

 template<class T> class shared_ptr; // freestanding-deleted

-end example]

Changes in [compliance]

Add new rows to the "C++ headers for freestanding implementations" table:

Subclause		Header(s)
[...]	[...]	[...]
?? [expected]	Expected objects	<expected>
?? [views.contiguous]	Contiguous access	
?? [views.multidim]	Multidimensional access	<mdspan>
[...]	[...]	[...]

Changes in [version.syn]

Drafting note: __cpp_lib_mdspan is intentionally left unmodified.

```
#define __cpp_lib_freestanding_expected new-val // freestanding, also in <expected>
#define __cpp_lib_freestanding_mdspan new-val // freestanding, also in <mdspan>
...
#define __cpp_lib_out_ptr 202106L new-val // freestanding, also in <memory>
...
#define __cpp_lib_span 202002L new-val // freestanding, also in <span>
```

Changes in [memory.syn]

Instructions to the editor:

Please append a *// freestanding-deleted* comment to the following declaration:

- `shared_ptr`

Please append a *// freestanding* comment to the following declarations:

- `out_ptr_t`
- `out_ptr`
- `inout_ptr_t`
- `inout_ptr`

Changes in [\[expected.syn\]](#)

Instructions to the editor:

Please insert a *// mostly freestanding* comment at the beginning of the [\[expected.syn\]](#) synopsis.

Please append a *// partially freestanding* comment to every expected declaration in the synopsis.

Changes in [\[expected.object.general\]](#)

Instructions to the editor:

Please append a *// freestanding-deleted* comment to every value overload in the class synopsis.

Changes in [\[expected.void.general\]](#)

Instructions to the editor:

Please append a *// freestanding-deleted* comment to every value overload in the class synopsis.

Changes in [\[span.syn\]](#)

There are two versions of this wording. One version assumes that `span.at()` has been added in [P2821] "`span.at()`", the other version assumes that it has not been added yet.

Changes assuming [P2821] "`span.at()`" has NOT been merged

Instructions to the editor:

Please insert an *// all freestanding* comment at the beginning of the [\[span.syn\]](#) synopsis.

Changes assuming [P2821] "`span.at()`" has been merged

Instructions to the editor:

Please insert a *// mostly freestanding* comment at the beginning of the [\[span.syn\]](#) synopsis.

Please append a *// partially freestanding* comment to the `span` declaration.

Changes in [\[views.span\]](#)

Drafting note: Only apply this change if [P2821] "`span.at()`" has been merged.

Please append a *// freestanding-deleted* comment to every overload of `at`.

Changes in [\[mdspan.syn\]](#)

Instructions to the editor:

Please insert an `// all freestanding` comment at the beginning of the [\[mdspan.syn\]](#) synopsis.

References

- [P0009R18] Christian Trott et al. 2022-07-13. MDSPAN.
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p0009r18.html>
- [P0122R7] Neil MacIntosh et al. 2018-03-16. span: bounds-safe views for sequences of objects.
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0122r7.pdf>
- [P0323R12] Vicente Botet et al. 2022-01-07. std::expected.
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p0323r12.html>
- [P1132R8] JeanHeyd Meneide et al. 2021-06-15. out_ptr - a scalable output pointer abstraction.
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p1132r8.html>
- [P2407R5] Ben Craig. 2023-07-26. Freestanding Library: Freestanding Library: Partial Classes.
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2338r4.html>
- [N4950] Thomas Köppe. 2023-05-10. Working Draft, Standard for Programming Language C++.
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf>